
HungerHeal

System design and scaling

150 questions with answers

12 sections

Event-driven microservices · Go · Kafka · Redis

HungerHeal — system design

A food-rescue platform built as event-driven microservices, whose core is a real-time multi-constraint matching engine.

This document is written to be **said out loud**. Section 1 is the answer to "tell me about your project", scripted at three lengths. Everything after it is the depth to draw on when they follow up.

Contents

§

1 Tell me about your project — the scripted answer

2 The problem, and why it is hard

3 Requirements

4 Scale model and capacity estimation

5 High-level architecture

6 The matching engine

7 Storage: why four different databases

8 Communication: why an event log

9 **Scaling, tier by tier**

10 Failure modes

11 Observability

12 Trade-offs I would defend

13 The whiteboard script

1 — Tell me about your project

The 20-second version

HungerHeal is a food-rescue platform. Restaurants and caterers post surplus food, and volunteer collection agents pick it up and take it to community kitchens. The interesting part is the matching: when a donation appears, the system decides in about a second which nearby volunteer should be asked to collect it, scoring every candidate on distance, equipment, current workload and track record. It is nine event-driven microservices, two of them in Go, communicating over Kafka.

Stop there. Let them pick the direction.

The 60-second version — use this one by default

HungerHeal is a food-rescue platform. Restaurants, canteens and event caterers post surplus food. Volunteer collection agents go on shift, get offered pickups near them, and take the food on to community kitchens.

The hard problem is matching, and it is a real-time one. When a donation is posted, the system has to decide — in seconds, not minutes — which of potentially hundreds of volunteers currently on shift should be asked to collect it. That is a multi-constraint decision: how far away they are, whether they have the equipment to carry that kind of food safely, how much they are already committed to, and how reliable they have been. And it is on a clock, because cooked food has a few hours of useful life, not days.

So I built a scoring engine. Every candidate is scored on four weighted factors, each normalised to 0–1 before weighting — distance is 35%, category fit 25%, current load 20%, rating 15%. The top three are offered the donation **in parallel**, and the first to accept wins. If nobody answers within the window, it re-scores over a wider radius, excluding whoever just ignored it.

Architecturally it is nine services over Kafka. The two on the hot path — live agent locations and the matching engine — are Go; the rest are Node. Live positions live in a Redis geo set so the "who is nearby?" query is sub-millisecond.

The thing I am proudest of is that every decision is explainable. The full score breakdown for every candidate is stored, winners and losers, so you can open any donation and see exactly why it went where it went.

The 2-minute version — only if they lean in

Add, in this order:

1. **The reliability story.** A donation is never lost because something downstream is down. donation-service saves it and publishes an event; if the broker is unreachable the donation is still accepted and a sweeper retries.
2. **The adding-services-is-free story.** Tracking, notifications and analytics were each added without changing a line of any existing service — a new consumer group receives the whole stream. Analytics was written last, read the log from the beginning, and reconstructed 133 events across four days including days before it existed.

3. **The one-line thesis.** *Distance is 35% of the decision, not 100% — a farther, properly equipped volunteer beats a closer, unequipped one, and I can demonstrate it.*

Three sentences never to say

- Anything vague: "it uses microservices for scalability." Say *what* scales and *what breaks first*.
 - "The algorithm decides." That is the answer the project exists to avoid.
 - Claiming the weights are tuned. They are reasoned heuristics; say so.
-

2 — The problem, and why it is hard

Surplus food is abundant and nearly worthless **as a logistics problem it is left unsolved**. A tray of cooked biryani is worth a great deal for about four hours and nothing at all after that. So the system's entire job is to compress the time between *"this food exists"* and *"somebody is on their way to get it"*.

That turns out to be four hard problems at once:

1. It is a real-time assignment problem. Hundreds of volunteers may be on shift. Which one should be asked? Naively "the nearest" is wrong — a volunteer 200m away on a bicycle with no insulated box cannot carry forty hot meals.

2. It is multi-constraint. Distance, equipment, current workload and reliability all matter, and they are in different units — kilometres, booleans, counts, a rating out of five. They have to be made comparable before they can be combined.

3. It has a deadline, and the deadline depends on the food. Hot food needs an answer in ninety seconds. Tinned goods can wait five minutes and be matched more carefully.

4. Nobody can be forced to accept. A volunteer is not an employee. The system can only *offer*, which means it must handle refusals, silence, and races — because if you offer to three people at once, two of them lose.

A single-process CRUD application can store donations. It cannot do this.

3 — Requirements

Functional

Donors	post surplus food with category, quantity, pickup point, best-before, photos
Agents	go on shift, share live location, declare equipment, accept/decline offers, mark collected
Matching	automatic, within seconds, no human in the path
Tracking	both sides see live status and a full timeline
Explainability	every scoring decision is retrievable, including losing candidates
Analytics	historical record of every event, queryable by day and by donation

Non-functional — the ones that shaped the design

Requirement	Target	What it forced
Match latency	< 2s from post to offer	Redis geo query, not a database scan
Durability of a donation	never lost	transactional outbox
Correctness under redelivery	exactly-once <i>effect</i>	idempotent consumers keyed on eventId
Only one winner per donation	strict	atomic claim lock
Independent deployability	each service ships alone	database per service, events not calls
Auditability	every decision reconstructable	score breakdowns persisted, full event log

Explicitly out of scope

Payments. Routing and turn-by-turn navigation. Push/SMS delivery (the priority flag is carried; nothing delivers it). Multi-tenancy. These are named because an unbounded scope is a worse answer than a bounded one.

4 — Scale model and capacity estimation

Numbers an interviewer can check. Two tiers: what it is designed for, and what would break at 100×

Tier 1 — one city

registered donors	50,000
registered agents	5,000
agents concurrently ON SHIFT	2,000
donations per day	20,000
peak donation rate (meal-time burst)	~2.2 / sec

Derived load

```
LOCATION WRITES
2,000 on-shift agents ÷ 20s reporting interval = 100 writes/sec
each write = GEOADD + SET(alive), pipelined    = 200 Redis ops/sec

MATCHING (per donation)
1 × GEOSEARCH
up to MAX_CANDIDATES (100) × (HGETALL caps + GET load), pipelined
≈ 201 Redis ops per donation
at 2.2 donations/sec                                = ~440 Redis ops/sec

OFFERS + DEDUP (per donation)
SETNX dedup, 3 offer writes, ZADD deadline, ... = ~8 ops
at 2.2 donations/sec                                = ~18 Redis ops/sec

DEADLINE WATCHER
1 × ZRANGEBYSCORE every 5s                        = 0.2 Redis ops/sec

TOTAL                                              ≈ 660 Redis ops/sec
```

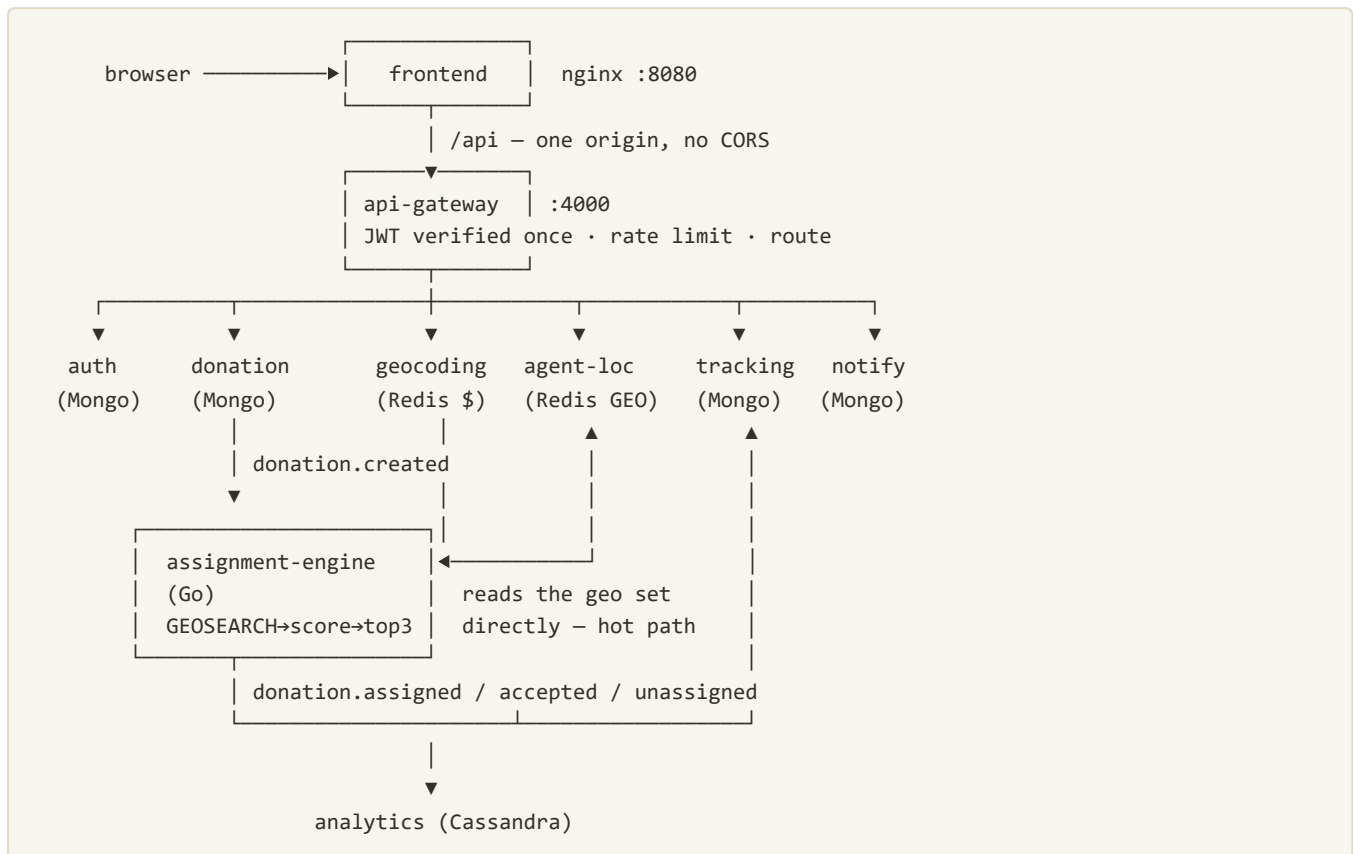
A single Redis handles **100,000+ simple ops/sec**. So tier 1 runs at roughly **0.7% of one Redis instance** — about 150× headroom.

The honest framing: at this scale nothing is under strain. The interesting question is not "is it fast enough" but "**what breaks first when it is not?**" — which is section 9.

Other tiers at tier-1 load

Kafka	2.2 donations/sec × ~4 events each ≈ 9 events/sec (Kafka does millions/sec — not close to a limit)
MongoDB	~2 writes/sec, a few hundred reads/sec from dashboard polling (trivial)
Cassandra	~9 appends/sec (trivial, but see §9 for the partition-key problem)

5 — High-level architecture



Everything between services is Kafka. No service calls another to tell it something happened. The two synchronous calls that exist are genuine questions: donation-service asks geocoding-service to resolve an address, and agent-location-service asks auth-service for an agent's capabilities once.

The one deliberate exception: assignment-engine reads the Redis keys agent-location-service writes, rather than calling its API. An HTTP hop would add latency to a query Redis answers in under a millisecond, on the one path where latency costs food quality. The cost is that the key layout is now a contract between two services — documented in both, asserted by both test suites.

6 — The matching engine

The score

```
score = 0.35 × distance      (1 - distanceKm / searchRadiusKm)
       + 0.25 × categoryFit  (compatibility matrix, floor 0.35)
       + 0.20 × load         (1 / (1 + pendingPickups))
       + 0.15 × rating       (rating / 5)
```

Every term is normalised to 0–1 **before** its weight is applied. Without that, distance in kilometres and a count of pickups are incomparable, and whichever had the larger numeric range would dominate regardless of the weight beside it.

The weights sum to 0.95, not 1.0 — deliberately kept as specified rather than rescaled. It changes no ranking, because every candidate is scaled by the same constant. It only matters when a human reads a score, which is why the UI always shows `0.9022 / 0.95`.

Hard filters, not penalties

Two things exclude a candidate *before* scoring: not accepting work, and not carrying that food category.

A penalty only lowers a score; it does not remove anyone. So a penalised candidate still wins when they are the only one — which is precisely the case where it matters most that they cannot do the job.

Why a better-equipped agent scores *lower* on tinned goods

Equipment	Cooked	Perishable	Bakery	Packaged
Insulated + refrigerated	1.00	1.00	0.70	0.55
Insulated only	1.00	0.75	0.75	0.60
Neither	0.40	0.45	1.00	1.00

This looks backwards until you see it as **allocating a scarce resource**. If equipped volunteers won everything, the one hot meal that genuinely needs an insulated box would find them all busy carrying tins.

Nothing scores zero: a zero would make every candidate score zero in an area with no equipped volunteers, and the ranking would collapse into noise exactly when you most need a sensible answer.

Urgency sets time, not rank

Urgency is a property of the *donation*, identical for every candidate — so as a score term it would shift everyone equally and change nothing. It sets the clock instead:

Urgency	Categories	Window	Radius ladder
HIGH	cooked, perishable raw	90s	5 → 12 → 25 → 50 km
MEDIUM	bakery, beverages	3 min	5 → 10 → 20 → 35 km
LOW	packaged	5 min	5 → 8 → 12 → 20 km

The ladders differ in **shape**, not just size. High urgency jumps far and fast — a volunteer 25km away who says yes *now* beats a perfect match found after four patient rounds. Low urgency widens gently, because tins keep and it is worth finding someone genuinely close.

The offer race

Three are offered **in parallel**, because asking in sequence spends the food's remaining life on politeness — three 90-second waits is four and a half minutes to reach the third person.

Exactly one can win. `SETNX donation:<id>:claimed` is atomic, so the first command to reach Redis wins and the others get 409. **Losing a race you were invited into is a normal outcome, not an error** — the UI says "another agent accepted this one first".

7 — Storage: why four different databases

The question behind this is "did you pick these deliberately or collect buzzwords?" — so each one needs a reason and a cost.

Store	Holds	Why this one	What it costs
MongoDB	users, donations, status, notifications	documents fit: nested quantity, nested location, variable fields by category; 2dsphere indexes	no cross-service joins or FK integrity — the agent's name is copied onto the tracking record
Redis	live positions, load, offers, geocode cache	four different jobs, only one of which is caching	one shared hot dependency; it is the system of record for position, so losing it stops matching
Kafka	the event log	replay, and independent consumers	at-least-once delivery, which every consumer must handle
Cassandra	historical events	append-only, ever-growing time series; linear write scaling	you must know your queries before designing tables — the same events are stored twice

Redis is doing four genuinely different jobs

agents:live	GEO set	"who is within 5km?" in sub-ms
agent:<id>:load	counter	INCR/DECR are atomic; read-modify-write is not
geocode:<key>	string+TTL	an address does not move – 30-day TTL
donation:<id>:claimed	SETNX	"am I first?" answered atomically

Worth naming: **a sorted-set member cannot expire** — only a whole key can. A geo set is a sorted set underneath, so there is no way to say "forget this position in two minutes". Solved with a separate heartbeat key that *does* expire, checked on every read, plus a reaper that removes orphaned geo members so the set cannot grow without bound.

The Cassandra duplication is the design, not a mistake

`donation_events` is partitioned by `donation_id` and answers "everything about donation X". It **cannot** answer "what happened today?" without scanning every partition. So `events_by_day` stores the same events again, partitioned by day.

In a relational database that is a normalisation error. In Cassandra, **you duplicate data to buy query performance** — that is the trade the model asks you to make explicitly.

8 — Communication: why an event log

The argument in one sentence: a donation is never lost because something downstream is down.

With a direct HTTP call, donation-service fails when the matching engine is restarting, and the donor is told to try again — for something the system could simply have accepted. With an event log, the donation is recorded as a fact and the engine reads it whenever it comes back.

The dual-write problem, and the outbox

Creating a donation means writing to Mongo **and** publishing to Kafka, with no transaction spanning both. A crash in between leaves a donation nothing downstream ever hears about — the donor sees success and their food is never collected. The worst kind of failure, because it is silent.

```
① save with eventPublished: false      ← durable first, one atomic write
② publish to Kafka
③ on success, set eventPublished: true
④ a sweeper every 15s republishes anything still false
```

Verified by stopping Kafka, posting a donation (still 201), restarting, and watching the sweeper publish it under the request's **original** trace id.

At-least-once, and the two halves that make it safe

Consumers **process then commit**, so a crash redelivers rather than loses. That means duplicates are guaranteed, so every consumer deduplicates on `eventId` with `SETNX`, namespaced per service.

With one subtlety: the marker is set *before* the work, so **a failure must release it** — otherwise the retry skips an event that was never processed.

Why not exactly-once? It needs Kafka transactions across the consume-process-produce cycle plus a transactional sink. The work here ends in Mongo and Redis, so idempotent handlers get the same practical result for far less machinery.

Keyed by donationId

Kafka guarantees ordering **within a partition**, not across a topic. Keying by `donationId` puts a donation's whole lifecycle on one partition, so `assigned` can never be processed after `accepted`. This is also what makes the engine horizontally scalable — see next section.

9 — Scaling, tier by tier

The section that matters. Structure every answer as: **what scales trivially, what scales with work, what breaks first, and what I would do about it.**

9 1 — The stateless tier: trivial

`api-gateway`, `auth`, `donation`, `geocoding`, `tracking`, `notification` hold no state between requests. Add instances behind a load balancer; they scale linearly until their datastore becomes the limit.

One caveat I would volunteer: the gateway's rate limiter uses in-memory counters, so with N gateway instances the effective limit is $N \times$ the configured value. Fixing it means moving the counters to Redis — which is already there.

9 2 — assignment-engine: scales by Kafka partitions

This is the one people expect to be hard, and it is already solved by the keying decision.

```
topics are created with 3 partitions
events are keyed by donationId
↓
all events for ONE donation land on ONE partition, in order
↓
a consumer group assigns each partition to exactly ONE instance
↓
3 partitions ⇒ up to 3 engine instances, each owning a disjoint
set of donations, with per-donation ordering preserved
```

To scale further: **more partitions**. 30 partitions allows 30 instances.

The caveat worth knowing: adding partitions changes the key→partition mapping, so a donation mid-flight could have its later events land on a different partition and be processed out of order. So you either over-provision partitions up front, or repartition during a quiet window. I would over-provision — partitions are cheap, rebalancing is not.

The deadline watcher is already safe to run on every instance. Each one polls the same sorted set and sees the same due deadlines, but it claims one with `ZREM`, which returns 1 for exactly one caller. One expiry produces one timeout event however many instances are running.

9 3 — Redis: the real bottleneck, and why the domain saves us

At tier 1 Redis runs at ~0.7% capacity. At 100× — a national deployment, 200,000 agents on shift, 220 donations/sec — the arithmetic changes:

location writes	$10,000/\text{sec} \times 2 \text{ ops}$	=	20,000 ops/sec
matching	$220/\text{sec} \times 201 \text{ ops}$	=	44,200 ops/sec
offers + dedup	$220/\text{sec} \times 8 \text{ ops}$	=	1,760 ops/sec
TOTAL		≈	66,000 ops/sec

That is at the ceiling of a single instance. **But the op count is not even the real problem** — this is the bit worth saying:

*GEOSEARCH is roughly $O(N + \log M)$, where M is the size of the set. With 200,000 members in **one** geo set, every single match pays for the size of the entire country's agent population — even though it only cares about 5km.*

The fix is the nicest property of this domain: it partitions geographically.

```
agents:live          →  agents:live:{region}
  one global set      →  one set per city / geohash prefix
```

A donation in Bengaluru only ever queries the Bengaluru shard. There is **no cross-shard query**, because the domain has no cross-city matching — nobody drives 600km for a biryani. So:

- each `GEOSEARCH` scans a set two orders of magnitude smaller;
- the shards can live on **different Redis instances**, so op rate divides too;
- routing is a pure function of the donation's coordinates — no lookup table.

Second split, orthogonal to the first: Redis is currently doing four jobs in one instance. Before sharding geographically I would separate them by concern — geo, cache, and offers/dedup — because they have completely different durability and eviction needs. The geocode cache can be evicted freely; a claim lock cannot.

9 4 — agent-location-service: the highest write rate

Every on-shift agent, every 20 seconds, forever. At 200,000 agents that is 10,000 writes/sec, which is why it is in Go.

It is stateless, so it scales horizontally. The load lands on Redis, addressed above.

The lever most people miss: the reporting interval is a direct multiplier on the write rate. It is 20s against a 120s TTL, giving six chances to survive a lapse. Moving to 30s cuts the write rate by a third and still gives four. That is a cheaper scaling win than any infrastructure change — but it trades against position freshness, and distance is 35% of the score.

9 5 — MongoDB

Writes are negligible at any realistic tier. Reads dominate, because dashboards poll.

```
step 1  read replicas – dashboard reads are tolerant of slight staleness
step 2  shard donations by donorId, tracking by donationId
        (both are the natural access key, so no scatter-gather)
step 3  replace polling with push ($9.7) – removes most reads entirely
```

9 6 — Cassandra: a real flaw at scale, and I would name it

Writes are append-only and Cassandra scales linearly by adding nodes. But one table has a problem I would raise before an interviewer found it:

donation_events	PRIMARY KEY ((donation_id), occurred_at, event_id)	✓ good
events_by_day	PRIMARY KEY ((day), occurred_at, event_id)	✗ hot
assignment_outcomes	PRIMARY KEY ((day), occurred_at, donation_id)	✗ hot
daily_totals	PRIMARY KEY (day)	✗ hot

Partitioning by day means every write today lands in one partition, so one replica set takes the entire cluster's write load while the other nodes idle. That is a textbook hot-partition anti-pattern. It is invisible at current volume and would be the first thing to fail under real traffic.

The fix is bucketing:

```
PRIMARY KEY ((day, bucket), occurred_at, event_id)
            bucket = hash(donation_id) % 16
```

Sixteen partitions per day instead of one, spread across the ring. Reading a whole day then means querying 16 partitions and merging — which is exactly the trade Cassandra expects you to make, and it is a `IN` clause, not a scan.

9 7 — The frontend: the weakest link, honestly

The UI **polls** — 10s for the donor dashboard, 5s for agent offers. At 200,000 on-shift agents that is 40,000 requests/sec of mostly-empty responses.

This is the part of the system I would change first at scale. WebSockets or Server-Sent Events would cut it to near zero, because the interesting events are already on Kafka — a push gateway subscribing to `donation.assigned` and fanning out to connected agents is a natural fit.

Polling is what I built because it is simple, stateless, and works through the proxy with no extra infrastructure. For a 90-second decision window a 5-second poll is adequate. I would not defend it beyond that.

9 8 — Scaling summary

Tier	Scales by	Breaks first at	Fix
Stateless services	add instances	—	shared rate-limit counters
assignment-engine	Kafka partitions	3 instances (current config)	more partitions, over-provisioned
Redis	—	one global geo set, ~66k ops/sec	shard by region; split by concern
agent-location	add instances	Redis behind it	longer report interval; geo shards
MongoDB	replicas, then shard	read load from polling	replicas → push instead of polling
Cassandra	add nodes	hot day-partition	bucket the partition key
Frontend	CDN	40k req/s of empty polls	WebSockets/SSE

If asked "what is the single bottleneck?" — the one global Redis geo set, and the answer is that the domain shards geographically for free.

10 — Failure modes

Being able to answer "what happens when X dies?" differently for each X is what shows the design is real.

What dies	What happens	Why
assignment-engine	donations still accepted, matched on restart	the event waits in Kafka
Kafka	donations still accepted (201), published later	the outbox
Redis	matching stops	it is the system of record for position
auth-service	existing tokens keep working; no new logins	a signed JWT is self-contained
geocoding-service	donations accepted un-geocoded	best-effort call, not a dependency
a consumer crashes mid-event	redelivered, deduplicated, no double-assign	process-then-commit + SETNX
analytics-service	nothing — it is optional	separate Docker profile; gateway returns 502 with an explanation

Redis is the single point of failure and I would say so plainly. In production: replication with automatic failover. What makes that acceptable rather than alarming is that **no durable data lives only in Redis** — positions are re-reported every 20 seconds, capabilities are a mirror of auth-service, and load counters can be rebuilt from tracking records. Losing Redis costs availability, not data.

The one I am not happy with

An unparseable message is logged and **committed**. The alternative is not committing, which means Kafka redelivers it forever and **blocks the partition**, taking down every donation behind it. So the choice is lose one message or lose the topic. The right answer is a dead-letter topic, and it is the top item on my list.

11 — Observability

Distributed tracing, the cheap version. One `x-trace-id`, generated at the browser and **reused, never regenerated**, by every service it touches:

```
docker compose logs | grep <traceId>
```

follows one donation from the browser, through the gateway, into Mongo, onto Kafka, through the Go engine, into the status service. The trace id is shown on the monitoring detail view precisely so an operator can copy it.

A service that generated a *new* id would break the chain at exactly the point you care about — which is why the outbox publishes under the **original** request's id even minutes later.

Structured JSON logs throughout, with the trace id bound to the context.

What is missing, and when I would add it: OpenTelemetry spans, so you get timings and a waterfall rather than grep; and RED metrics per service with alerting. At nine services grep is enough. At thirty it would not be.

The product-level metric that matters most is first-round match rate — how often the first batch of three says yes. It is the most direct evidence that the scoring picks the right people.

12 — Trade-offs I would defend

Decision	Alternative	Why this one
Events, not HTTP, between services	direct calls	a donation survives any downstream outage; new consumers are free
At-least-once + idempotency	exactly-once	same practical result, far less machinery, no transactional sink needed
Engine reads Redis directly	call agent-location's API	sub-ms vs an HTTP hop, on the one latency-critical path
Hard filters before scoring	penalties	a penalty still lets an unsuitable candidate win when they are the only one
Hand-picked weights	learned	there was no accept/reject history to learn from; it is now being collected
Straight-line distance	routing API	a per-candidate network call on the hot path
Polling	WebSockets	simple and stateless; the honest weak point
Two languages	all Node	Go where concurrency and latency are the problem; Node where it is plumbing

What I would change first, in order

1. **A dead-letter topic.** The current behaviour is the lesser of two bad options.
2. **Bucket the Cassandra day partitions.** A known hot partition.
3. **Push instead of polling.** Removes most read load in one change.
4. **Learn the weights.** The data is being collected now.

13 — The whiteboard script

Three minutes, drawn in this order. Narrate while you draw.

Step 1 — the actors. Two boxes at the edges: *Donor* and *Agent*. Say the problem in one line: "surplus food, and a volunteer who has to be found in seconds."

Step 2 — the write path. Gateway → donation-service → Mongo. Say: *"saved first, published second — that is the outbox, and it is why a donation is never lost."*

Step 3 — the log. Draw Kafka as a horizontal bar, not a box. Say: *"a log, not a queue. Several services read the same events independently."*

Step 4 — the engine. Below Kafka. Draw an arrow from it to a Redis cylinder labelled `agents:live`. Say: *"geo query for who is nearby, score them, offer to the top three in parallel."* **This is where you slow down** — it is the point of the whole system.

Step 5 — the fan-out. Three arrows up from Kafka to tracking, notification, analytics. Say: *"each was added without changing any existing service."*

Step 6 — only if asked to scale it. Draw the geo cylinder splitting into three labelled by city. Say: *"the bottleneck is this one set, and the domain shards geographically for free — nobody drives across the country for a donation."*

The four sentences to have ready verbatim

1. *"Distance is 35% of the decision, not 100%."*
2. *"Saved first, published second — a donation is never lost because something downstream is down."*
3. *"Three are offered in parallel; one atomic lock decides the winner. Losing that race is a normal outcome, not an error."*
4. *"Every scoring decision is stored, including the candidates who lost — so a bad match is diagnosable, not arguable."*